

Chapter 8: UNORDERED_SET

This is one of the most important STL containers for interviews.

If Set is:

Sorted + Unique

then Unordered Set is:

Unique + Very Fast

What Problem Does unordered_set Solve?

Suppose you want to answer:

Does x exist?

millions of times.

Using vector:

$O(n)$

Using set:

$O(\log n)$

Using unordered_set:

$O(1)$ average

Much faster.

Internal Implementation

Uses:

Hash Table

Not a tree.

This is why elements are NOT sorted.

Header

```
#include <unordered_set>
```

Declaration

```
unordered_set<int> us;
```

Properties

Unique Elements

1 2 2 3 3

Stored:

1 2 3

Not Sorted

Input:

```
10 5 20 1
```

Output may be:

```
20 1 10 5
```

or

```
5 20 1 10
```

or anything else.

Never rely on order.

Insert

insert()

```
us.insert(10);  
us.insert(5);  
us.insert(20);
```

Duplicate ignored:

```
us.insert(10);
```

Still only one 10.

Complexity

Average:

```
O(1)
```

Worst:

$O(n)$

(rare)

Print

```
for(auto x:us)
{
    cout<<x<<" ";
}
```

Order is unpredictable.

Find

Most used operation.

```
auto it = us.find(10);
```

Check:

```
if(it != us.end())
{
    cout<<"Found";
}
```

Complexity

Average:

$O(1)$

Count

```
us.count(10);
```

Returns:

```
1
```

or

```
0
```

Since duplicates are not allowed.

Complexity

```
O(1)
```

Average.

Erase

By Value

```
us.erase(10);
```

By Iterator

```
auto it = us.find(20);  
  
us.erase(it);
```

Complexity

```
O(1)
```

Average.

Size

```
us.size();
```

Empty

```
us.empty();
```

Clear

```
us.clear();
```

Swap

```
us1.swap(us2);
```

Time Complexity Table

Operation	Average	Worst
insert	O(1)	O(n)
erase	O(1)	O(n)
find	O(1)	O(n)
count	O(1)	O(n)

Interviewers care about average complexity.

Set vs Unordered Set

Feature	Set	Unordered Set
Structure	Red Black Tree	Hash Table
Order	Sorted	No Order
Search	$O(\log n)$	$O(1)$
Insert	$O(\log n)$	$O(1)$
lower_bound	Yes	No
upper_bound	Yes	No

Important Difference

Set:

```
set<int> s={10,20,30,40};  
  
s.lower_bound(25);
```

Works.

Unordered Set:

```
unordered_set<int> us;
```

No:

```
lower_bound()  
upper_bound()
```

✗ Not available.

Because data is not sorted.

Bucket Concept

Advanced Interview Topic.

Hash Table stores data in buckets.

Example:

```
Bucket 0 -> 10,20  
Bucket 1 -> 31  
Bucket 2 -> 42,72
```

Useful Functions

bucket_count()

Returns total buckets.

```
cout<<us.bucket_count();
```

load_factor()

Average elements per bucket.

```
cout<<us.load_factor();
```

rehash()

Increase bucket count.

```
us.rehash(100);
```

Rarely used.

Common Interview Uses

Contains Duplicate

Input:

```
1 2 3 1
```

Use:

```
unordered_set<int>
```

Fast Membership Test

```
if(us.count(x))
```

Remove Duplicates

```
unordered_set<int> us(v.begin(),v.end());
```

Longest Consecutive Sequence

LeetCode #128

Best solution uses:

```
unordered_set
```

Example

Contains Duplicate

```
bool containsDuplicate(vector<int>& nums)
{
    unordered_set<int> us;

    for(int x:nums)
    {
```

```
        if(us.count(x))
            return true;

        us.insert(x);
    }

    return false;
}
```

Complexity:

$O(n)$

Practice Questions

Easy

Q1

Insert:

10 20 10 30

Print size.

Output:

3

Q2

Check if 20 exists.

Q3

Remove 10.

Q4

Print all unique elements.

Q5

Count distinct elements.

Input:

1 2 2 3 1

Output:

3

Medium

Q6

Remove duplicates from vector.

Q7

Check if array contains duplicate.

Q8

Find intersection of two arrays.

Q9

Find union of two arrays.

Q10

Find first repeated element.

Interview Level

Q11

Contains Duplicate

LeetCode #217

Q12

Longest Consecutive Sequence

LeetCode #128

Q13

Happy Number

LeetCode #202

Q14

Intersection of Two Arrays

LeetCode #349

Q15

Find Missing Number

LeetCode #268

Common Mistakes

Mistake 1

Assuming sorted order.

```
unordered_set<int> us;
```

Never sorted.

Mistake 2

Using:

```
lower_bound()  
upper_bound()
```

✗ Not available.

Mistake 3

Using indexing.

```
us[0]
```

✗ Not allowed.

Revision Sheet

Header

```
#include <unordered_set>
```

Declaration

```
unordered_set<int> us;
```

Insert

```
us.insert(x);
```

Remove

```
us.erase(x);
```

Search

```
us.find(x);  
us.count(x);
```

Utilities

```
us.size();  
us.empty();  
us.clear();  
us.swap();
```

Hash Table Functions

```
bucket_count()  
load_factor()  
rehash()
```

Complexity

```
insert  O(1)  
erase   O(1)  
find    O(1)  
count   O(1)
```

Average case.

Interview Rule

Use:

```
set
```

When:

- Need sorting
- Need lower_bound
- Need upper_bound
- Need predecessor/successor

Use:

```
unordered_set
```

When:

- Need only existence checking
- Need fastest lookup
- Don't care about order

Mental Trigger

When you see:

- Contains Duplicate
- Fast Lookup
- Membership Test
- Remove Duplicates
- Hashing

Think:

```
unordered_set
```

Assignment

Solve:

1. Contains Duplicate.
 2. Count Distinct Elements.
 3. Union of Arrays.
 4. Intersection of Arrays.
 5. First Repeated Element.
 6. Missing Number.
 7. Happy Number.
 8. Longest Consecutive Sequence.
-